

TechHub — SRE Standards, Operations & Delivery

Document: 03 — SRE Standards, Operations & Delivery **Version:** 1.0.0 **Owner:** Member 3 (SRE / Platform Lead) · Member 4 (QA & Documentation Lead) **Status:** Active

Table of Contents

- 1. Kafka Topic & Event Governance
 - 2. API Contract Governance
 - 3. Monitoring and Observability Strategy
 - 4. Logging Strategy
 - 5. Security Best Practices
 - 6. Testing Strategy
 - 7. Integration Strategy
 - 8. Incident Management Workflow
 - 9. Sprint Execution Strategy
 - 10. Daily Standup Structure
 - 11. Shared Documentation Structure
 - 12. Definition of Done
 - 13. Risk Management Plan
 - 14. Production Readiness Checklist
 - 15. Final Delivery Checklist
-

1. Kafka Topic & Event Governance

1.1 Topic Naming Convention

techhub.<domain>.<entity>.<event-type>

Elements:

- techhub → Project prefix – always present
- <domain> → Service domain (users, events, projects, teams, notifications, community)
- <entity> → Business entity affected (user, event, project, team, post)
- <event-type> → Past-tense action (created, updated, deleted, joined, published, cancelled)

Examples:

techhub.users.user.created

```
techhub.events.event.created
techhub.projects.project.updated
techhub.teams.member.joined
techhub.community.post.published
```

1.2 Topic Registry

Topic	Producer	Consumer(s)	Owner
<code>techhub.users.user.created</code>	User Service	Notification Service	Member 1
<code>techhub.users.user.updated</code>	User Service	Team Service	Member 1
<code>techhub.users.user.deleted</code>	User Service	All relevant services	Member 1
<code>techhub.events.event.created</code>	Event Service	Notification Service	Member 2
<code>techhub.events.event.updated</code>	Event Service	Notification Service	Member 2
<code>techhub.events.event.cancelled</code>	Event Service	Notification Service	Member 2
<code>techhub.projects.project.created</code>	Project Service	Notification Service Team Service	Member 2
<code>techhub.projects.project.updated</code>	Project Service	Team Service	Member 2
<code>techhub.teams.team.created</code>	Team Service	Notification Service	Member 3
<code>techhub.teams.member.joined</code>	Team Service	Notification Service	Member 3
<code>techhub.teams.member.left</code>	Team Service	Notification Service	Member 3
<code>techhub.community.post.published</code>	Community Service	Notification Service	Member 4

1.3 Standard Event Payload Schema

All Kafka messages must follow this envelope:

```
json
{
  "eventId": "550e8400-e29b-41d4-a716-446655440000",
  "eventType": "techhub.users.user.created",
  "version": "1.0",
  "timestamp": "2026-05-01T10:30:00.000Z",
  "source": "user-service",
  "correlationId": "req-abc-123",
  "payload": {
    "userId": "usr-001",
    "email": "user@example.com",
    "username": "john_doe"
  }
}
```

Field	Type	Required	Description
eventId	UUID v4	✓	Unique event identifier
eventType	String	✓	Full topic-style event name
version	String	✓	Schema version (for forward compatibility)
timestamp	ISO 8601	✓	UTC event creation time
source	String	✓	Producing service name
correlationId	String	Recommended	For request tracing
payload	Object	✓	Domain-specific data

1.4 Consumer Group Naming

```
<consuming-service>-<topic-entity>-consumer-group
```

Examples:

```
notification-service-user-consumer-group
team-service-project-consumer-group
notification-service-event-consumer-group
```

1.5 Kafka Producer Configuration

properties

```
spring.kafka.producer.bootstrap-servers=${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSer
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.
spring.kafka.producer.acks=all
spring.kafka.producer.retries=3
spring.kafka.producer.properties.retry.backoff.ms=1000
```

1.6 Kafka Consumer Configuration

properties

```
spring.kafka.consumer.bootstrap-servers=${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
spring.kafka.consumer.group-id=${KAFKA_GROUP_ID}
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.enable-auto-commit=false
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringD
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serialize
spring.kafka.consumer.properties.spring.json.trusted.packages=*
spring.kafka.listener.ack-mode=MANUAL_IMMEDIATE
```

1.7 Event Contract Change Policy

Kafka events are consumed by multiple services. Breaking changes require:

1. Announcement to team 48 hours in advance
2. New version field value (`"version": "2.0"`) — never remove fields, only add
3. Consumer services updated in the same sprint before the producer is deployed
4. Old version maintained in parallel for one sprint (grace period)

2. API Contract Governance

2.1 Contract Definition Process

- Step 1 Service owner drafts OpenAPI 3.0 spec in docs/contracts/api-contracts/
- Step 2 Team reviews and approves via PR (1 reviewer minimum)
- Step 3 Contract is frozen – represents the interface commitment
- Step 4 Consuming services implement against the frozen contract
- Step 5 Integration tests validate that the implementation matches the spec

2.2 OpenAPI Minimum Requirements

```
yaml
openapi: "3.0.3"
info:
  title: User Service API
  version: "1.0.0"
  description: Manages user accounts, authentication, and profiles.
  contact:
    name: Member 1
servers:
  - url: http://localhost:8081
    description: Local development
  - url: http://api-gateway:8080/api/users
    description: Via Gateway (Docker/K8s)
```

2.3 Swagger UI Access Registry

Service	Local Swagger URL
API Gateway	<code>http://localhost:8080/swagger-ui.html</code>
User Service	<code>http://localhost:8081/swagger-ui.html</code>
Event Service	<code>http://localhost:8082/swagger-ui.html</code>
Project Service	<code>http://localhost:8083/swagger-ui.html</code>
Team Service	<code>http://localhost:8084/swagger-ui.html</code>
Community Service	<code>http://localhost:8085/swagger-ui.html</code>
Notification Service	<code>http://localhost:8086/swagger-ui.html</code>

2.4 Spring Boot Swagger Configuration

```
java
```

```

@Configuration
public class SwaggerConfig {
    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("<Service Name> API")
                .version("1.0.0")
                .description("<Service description>"))
            .components(new Components()
                .addSecuritySchemes("bearer-jwt", new SecurityScheme()
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT")));
    }
}

```

2.5 Breaking Change Policy

A **breaking change** is any change that:

- Removes an existing endpoint
- Renames or removes a request/response field
- Changes a field's data type
- Changes required status of a field
- Changes an HTTP method or status code

Breaking changes require:

1. Team announcement + 48-hour notice
2. PR approved by 2 members
3. All consuming services updated in the same PR or sprint
4. Version bump in the OpenAPI spec

3. Monitoring and Observability Strategy

3.1 Observability Stack

Component	Technology	Owner
Metrics Collection	Prometheus	Member 3
Metrics Visualization	Grafana	Member 4

Component	Technology	Owner
Service Health	Spring Boot Actuator	Each member
Log Aggregation	Stdout → kubectl logs (academic scope)	Each member
Alerting	Prometheus Alertmanager	Member 3

3.2 Required Spring Boot Actuator Setup

properties

```
# Mandatory in all application.properties
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=always
management.endpoint.health.probes.enabled=true
management.metrics.export.prometheus.enabled=true
management.metrics.tags.application=${spring.application.name}
management.metrics.tags.environment=${SPRING_PROFILES_ACTIVE:local}
```

3.3 Prometheus Scrape Configuration

yaml

```
# k8s/infrastructure/monitoring/prometheus/configmap.yaml
scrape_configs:
  - job_name: 'api-gateway'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['api-gateway-svc:8080']

  - job_name: 'user-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['user-service-svc:8081']

  - job_name: 'event-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['event-service-svc:8082']

# Repeat for all services...
```

3.4 Mandatory Metrics (All Services)

Metric	Type	Purpose
<code>http_server_requests_seconds</code>	Histogram	Request duration by endpoint and status
<code>jvm_memory_used_bytes</code>	Gauge	JVM heap and non-heap usage
<code>jvm_gc_pause_seconds</code>	Summary	Garbage collection pause duration
<code>process_cpu_usage</code>	Gauge	CPU usage percentage
<code>spring_data_repository_invocations_seconds</code>	Histogram	Database query latency
<code>kafka_consumer_records_consumed_total</code>	Counter	Messages consumed (Kafka services)
<code>kafka_producer_record_send_total</code>	Counter	Messages produced (Kafka services)

3.5 Grafana Dashboard Ownership

Dashboard	Owner	Content
TechHub — System Overview	Member 4	All services health, error rates, latency
Gateway & Auth	Member 1	Request rates, auth failures, Redis cache hits
Business Services	Member 2	Event/Project metrics, Kafka consumer lag
Infrastructure	Member 3	Kafka, PostgreSQL, Redis, Zookeeper
Community & Notifications	Member 4	Community service metrics, notification delivery

3.6 Alerting Rules

Alert Name	Condition	Severity	Owner
<code>ServiceDown</code>	Pod not ready for > 2 minutes	SEV-1	Service owner
<code>HighErrorRate</code>	HTTP 5xx > 5% of requests over 5 min	SEV-2	Service owner
<code>HighLatency</code>	P99 latency > 2 seconds	SEV-2	Service owner
<code>KafkaConsumerLag</code>	Consumer lag > 1000 messages	SEV-2	Member 3

Alert Name	Condition	Severity	Owner
DatabaseConnectionHigh	Connection pool usage > 80%	SEV-3	Member 3
PodRestartLoop	Pod restarted > 3 times in 15 min	SEV-2	Service owner
HighMemoryUsage	Memory usage > 90% of limit	SEV-3	Service owner

4. Logging Strategy

4.1 Log Level Policy

Environment	Default Level	Format
Local (default profile)	DEBUG	Human-readable text
Docker Compose (docker profile)	INFO	Human-readable text
Kubernetes (k8s profile)	INFO	Structured JSON

4.2 Structured JSON Log Format (Kubernetes)

```
json
{
  "timestamp": "2026-05-01T10:30:00.000Z",
  "level": "INFO",
  "service": "user-service",
  "traceId": "abc123def456",
  "spanId": "xyz789",
  "thread": "http-nio-8081-exec-1",
  "logger": "com.techhub.user.service.UserService",
  "message": "User created successfully",
  "userId": "usr-001",
  "duration_ms": 42
}
```

4.3 Spring Boot Logging Configuration

```
properties
```

```
# application-k8s.properties
logging.level.root=INFO
logging.level.com.techhub=INFO
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level [%X{traceId}]
```

For JSON logging in Kubernetes, add `logstash-logback-encoder` to `pom.xml`:

```
xml

<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
  <version>7.4</version>
</dependency>
```

4.4 Logging Rules

Rule	Detail
Never log passwords or tokens	Security critical
Never log full PII	Compliance
Log INFO for state transitions	Request received, entity created, event published
Log WARN for recoverable errors	Retry attempts, fallback activated
Log ERROR with full stack trace	Unrecoverable failures only
Always include service name	Every log entry must identify its source
Include traceId when available	Enables cross-service tracing

5. Security Best Practices

5.1 Authentication Architecture

```
Internet
  |
  ▼
API Gateway ← JWT validation happens HERE
  |
  Services behind the gateway trust internal requests
```



Internal Services (no re-validation required)

- JWT secret stored in Kubernetes Secret, injected at runtime.
- Token expiration: **1 hour maximum**.
- Refresh token mechanism: handled by User Service.
- Services must not accept unauthenticated requests from external traffic.

5.2 Container Security Checklist

Requirement	Enforcement
Run as non-root user	Dockerfile: <code>RUN adduser</code> + <code>USER</code> instruction
Use Alpine base images	Minimize attack surface
Pin base image versions	No <code>:latest</code> on base images
No secrets baked into images	Runtime env vars only
Single exposed port	One port per container
Read-only filesystem (optional)	<code>readOnlyRootFilesystem: true</code> in K8s

5.3 Secret Management Rules

- ✓ Store here: Kubernetes Secrets
- ✓ Store here: Environment variables (non-sensitive only)
- ✓ Store here: ConfigMaps (non-sensitive configuration)
- ✗ Never store here: Git repository (any branch)
- ✗ Never store here: Docker images (any layer)
- ✗ Never store here: `application.properties` committed to Git
- ✗ Never store here: Docker Compose `.env` file committed to Git
- ✗ Never store here: GitHub Actions logs (mask secrets)

5.4 Network Security

- All inter-service communication uses **internal DNS names** (Kubernetes service names), never IPs.
- Only the API Gateway and Frontend have external-facing ports.
- All services communicate within the `techhub` namespace.
- Database services are never directly exposed externally.

5.5 Dependency Security

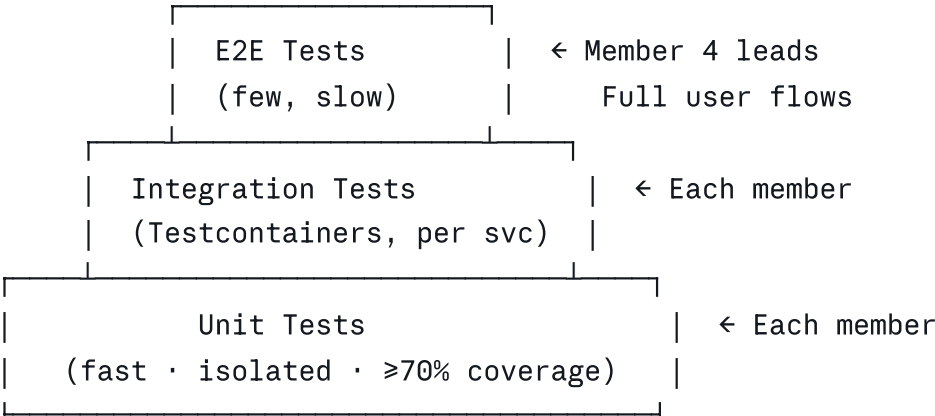
```
bash

# Check for known vulnerabilities in dependencies
mvn dependency:analyze
mvn org.owasp:dependency-check-maven:check

# Keep dependencies updated
mvn versions:display-dependency-updates
```

6. Testing Strategy

6.1 Test Pyramid



6.2 Testing Requirements per Member

Type	Member 1	Member 2	Member 3	Member 4
Unit Tests	✓ Required	✓ Required	✓ Required	✓ Required
Integration Tests	✓ Required	✓ Required	✓ Required	✓ Required
API Contract Tests	✓ Lead	✓	✓	✓
Kafka Contract Tests	✓ Producer	✓ Producer	✓ Consumer	✓
E2E Tests	Support	Support	Support	✓ Lead

6.3 Coverage Gate

CI pipelines enforce a minimum of **70% line coverage** per service. PRs that reduce coverage below this threshold are blocked.

6.4 Testcontainers Setup

xml

```
<!-- pom.xml: Testcontainers BOM -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.testcontainers</groupId>
      <artifactId>testcontainers-bom</artifactId>
      <version>1.19.3</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>kafka</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

6.5 Integration Test Example Structure

java

```

@SpringBootTest
@Testcontainers
class UserServiceIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:15-alpine");

    @Container
    static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.5.0"));

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
        registry.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
    }

    @Test
    void shouldCreateUserAndPublishEvent() {
        // Given
        // When
        // Then
    }
}

```

7. Integration Strategy

7.1 Contract-First Principle

All service integrations follow a contract-first approach:

1. DEFINE → Write the API or Kafka contract document
2. REVIEW → All consuming members review and approve
3. FREEZE → Contract is locked; changes require team approval
4. IMPLEMENT → Each service implements independently against the contract
5. VALIDATE → Integration tests confirm compliance

7.2 Inter-Service Communication Rules

Rule	Detail
No direct DB access	Services communicate only through APIs or Kafka
Use service DNS names	<code>http://user-service-svc:8081</code> — never hardcoded IP
Use environment variables	Never hardcode service URLs
Kafka for async operations	Use Kafka for events that don't need an immediate response
HTTP for synchronous calls	Use REST for synchronous request/response

7.3 Integration Sequence

Phase 1: Gateway → User Service	HTTP · JWT authentication
Phase 2: Gateway → All Services	HTTP routing verified end-to-end
Phase 3: User Service → Kafka	user.created events published
Phase 4: Notification Service ← Kafka	Consumes all domain events
Phase 5: Event/Project/Team → Kafka	Domain events flow correctly
Phase 6: Community Service	Full service mesh validated
Phase 7: Frontend → Gateway	Complete end-to-end user flows

7.4 Cross-Service Call Pattern (Spring Boot)

```
java

// application.properties
USER_SERVICE_URL=http://user-service-svc:8081

// Configuration class
@Value("${USER_SERVICE_URL}")
private String userServiceUrl;

// WebClient usage (preferred over RestTemplate)
@Bean
public WebClient userServiceClient() {
    return WebClient.builder()
        .baseUrl(userServiceUrl)
        .build();
}
```

8. Incident Management Workflow

8.1 Severity Levels

Level	Definition	Response Time	Who Responds
SEV-1	Complete platform outage — no service accessible	Immediate	All members
SEV-2	Major feature unavailable or data loss risk	15 minutes	Service owner + Member 3
SEV-3	Degraded performance or partial failure	1 hour	Service owner
SEV-4	Minor issue, workaround available	Next standup	Service owner

8.2 Incident Response Workflow

STEP 1 – DETECT

Alert fires via Prometheus Alertmanager
OR team member observes failure
→ Identify affected service and scope

STEP 2 – TRIAGE

Service owner investigates their service
Member 3 investigates shared infrastructure (Kafka, DB, Redis)
→ Assign severity level

STEP 3 – COMMUNICATE

Notify team in group channel with:

- What is broken
- Severity level
- Who is investigating
- ETA for first update

STEP 4 – MITIGATE

Option A: Quick fix deployed
Option B: Rollback to previous version
→ `kubectl rollout undo deployment/<service> -n techhub`

STEP 5 – RESOLVE

Root cause identified and addressed
Fix verified in running deployment
Team notified of resolution

STEP 6 – POST-MORTEM

Brief document in docs/sre/incidents/<date>-<service>-incident.md

Format:

- What happened
- Root cause
- Impact duration
- Resolution
- Action items to prevent recurrence

8.3 Incident Command Reference

```
bash

# Get pod status across namespace
kubectl get pods -n techhub

# Get events (ordered by time)
kubectl get events -n techhub --sort-by='.lastTimestamp'

# Tail logs from failing service
kubectl logs -f deployment/<service-name> -n techhub --previous

# Rollback immediately
kubectl rollout undo deployment/<service-name> -n techhub

# Verify rollback
kubectl rollout status deployment/<service-name> -n techhub

# Scale down a failing service (emergency isolation)
kubectl scale deployment/<service-name> --replicas=0 -n techhub

# Scale back up after fix
kubectl scale deployment/<service-name> --replicas=1 -n techhub
```

9. Sprint Execution Strategy

9.1 Sprint Timeline

Phase	Target Days	Focus
Kickoff & Scaffolding	Days 1-2	Project setup · Service skeletons · Local dev working
Core Development	Days 3-6	Feature implementation · Unit tests
Docker Integration	Days 5-7	Dockerfiles · Docker Compose integration starts

Phase	Target Days	Focus
Kubernetes Deployment	Days 7-10	Progressive K8s deployment (see Phase plan in Doc 02)
Integration & Kafka	Days 8-11	Service-to-service · Event flows verified
QA & Testing	Days 11-13	Integration tests · Load testing · Bug fixes
Final Delivery	Day 14	Validation · Documentation · Demo preparation

9.2 Key Sprint Rules

- ✓ Docker Compose integration begins on Day 5 – not Day 13.
- ✓ Kubernetes deployment begins on Day 7 – not the night before delivery.
- ✓ API contracts frozen by end of Day 3.
- ✓ Each service deployed to K8s before moving to the next.
- ✓ Daily integration validation – never leave integration to the last day.
- ✓ Every CI pipeline must be green before moving to the next sprint task.

9.3 GitHub Projects Board

Configure with these columns (Kanban flow):

Column	Definition
Backlog	All planned work, not yet started
To Do	Ready to begin this sprint
In Progress	Being actively developed
Code Review	PR open, awaiting review
Docker Ready	Service containerized, Compose working
K8s Ready	Service deployed and healthy in Kubernetes
Integrated	Service communicating with its dependencies
Tested	Unit + integration tests passing
Done	Merged, deployed, verified

10. Daily Standup Structure

10.1 Standup Format

Field	Value
Time	Daily · 09:00 (team agreement)
Duration	15 minutes maximum
Format	Synchronous (voice/video) or async written
Facilitator	Rotating · Member 3 default

10.2 Synchronous Standup Template

Each member answers in order:

1. COMPLETED (Yesterday)

What did I finish?
Did I merge or deploy anything?
Any CI pipeline failures I resolved?
2. IN PROGRESS (Today)

What am I working on?
Any code, Docker, or K8s tasks targeted?
3. BLOCKERS

Is anything blocking me?
Do I need another member's input?
Am I waiting on an API contract or dependency?
4. DEPLOYMENT STATUS

Are all my services healthy in Kubernetes?
Any health check failures or pod restarts?

10.3 Async Standup Format (Written — for remote sessions)

- [Member N — YYYY-MM-DD]
- ✓

Done:

<what was completed>

🔄

Today:

<what I'm working on today>

🚫

Blocked:

<blockers | None>

🚀

Status:

<all services healthy | specific issue>
- ## 10.4 When to Escalate Immediately
- Do not wait for the next standup. Notify the team immediately if:
- A CI pipeline is broken and blocking a PR merge

- A shared infrastructure service (Kafka, DB, Redis) is down
 - An API contract conflict is discovered
 - A Kubernetes pod is in CrashLoopBackOff for > 30 minutes
 - Integration between two members' services is failing
-

11. Shared Documentation Structure

```
docs/
|
├─ architecture/
|   ├── system-overview.md           ← High-level diagram · Owner: Member 4
|   ├── data-flow.md                 ← Data flows between services
|   ├── tech-decisions.md            ← Architecture Decision Records (ADRs)
|   └─ sequence-diagrams/            ← Request flow diagrams
|
├─ contracts/
|   ├── api-contracts/                ← OpenAPI specs per service
|   |   ├── gateway.yaml
|   |   ├── user-service.yaml
|   |   ├── event-service.yaml
|   |   ├── project-service.yaml
|   |   ├── team-service.yaml
|   |   ├── community-service.yaml
|   |   └─ notification-service.yaml
|   |
|   └─ kafka-contracts/                ← Event payload schemas
|       ├── user-events.md
|       ├── event-events.md
|       ├── project-events.md
|       ├── team-events.md
|       └─ community-events.md
|
├─ sre/
|   ├── 01_GIT_AND_GITHUB_GOVERNANCE.md    ← This suite: Doc 1
|   ├── 02_DEVOPS_K8S_DOCKER_GOVERNANCE.md ← This suite: Doc 2
|   ├── 03_SRE_STANDARDS_OPERATIONS.md     ← This suite: Doc 3
|   └─ incidents/                          ← Post-mortem reports
|
├─ deployment/
|   ├── local-setup.md                 ← How to run locally · Owner: Member 4
|   ├── docker-compose-guide.md        ← Compose instructions · Owner: Member 3
|   └─ kubernetes-guide.md             ← K8s deployment guide · Owner: Member 3
|
└─ uml/
```

└─ class-diagrams/
└─ entity-diagrams/
└─ use-case-diagrams/

11.1 Documentation Ownership

Document	Owner	Reviewer
System Overview	Member 4	All
API Contracts	Each service owner	Consuming member
Kafka Contracts	Producer owner	Consumer owner
SRE Governance (this suite)	Member 3	All
Local Setup Guide	Member 4	All
Kubernetes Guide	Member 3	Member 4
UML Diagrams	Member 4	All

12. Definition of Done

12.1 Service-Level DoD

A service is **Done** when ALL of the following are true:

Code Quality

- ☐ All assigned features implemented
- ☐ No hardcoded localhost, IPs, or passwords
- ☐ No commented-out dead code
- ☐ All configuration externalized to environment variables

API

- ☐ Swagger annotations on all endpoints
- ☐ OpenAPI spec in `docs/contracts/api-contracts/`
- ☐ DTOs validated with `@Valid`, `@NotNull`, etc.

Testing

- ☐ Unit test coverage $\geq 70\%$
- ☐ Integration tests cover critical flows
- ☐ All CI pipeline stages pass (green)

Docker

- ☐ Multi-stage Dockerfile follows standard template
- ☐ Image builds without errors
- ☐ `.dockerignore` present
- ☐ Container runs as non-root user
- ☐ `HEALTHCHECK` in Dockerfile

Docker Compose

- ☐ Service block added to `docker-compose.yml`
- ☐ Service starts clean with `docker compose up`
- ☐ Health check passes in Compose environment

Kubernetes

- ☐ `deployment.yml` with resource limits and probes
- ☐ `service.yml` with correct port mapping
- ☐ `configmap.yml` with all required env vars
- ☐ Pod reaches `Running/1/1` within 60 seconds
- ☐ Readiness and liveness probes both pass

Observability

- ☐ `/actuator/health` returns `{"status": "UP"}`
- ☐ `/actuator/prometheus` endpoint responds
- ☐ Logs are clean (no stack traces during normal operation)

Security

- ☐ No secrets in code, ConfigMaps, or committed files
- ☐ Container runs as non-root user
- ☐ Sensitive env vars reference Kubernetes Secrets

Documentation

- ☐ `README.md` in service folder with setup instructions
 - ☐ Swagger accessible at `/swagger-ui.html`
 - ☐ API contract updated in `docs/contracts/`
-

13. Risk Management Plan

13.1 Risk Register

Risk	Likelihood	Impact	Mitigation	Owner
One member owns all Kubernetes	High	High	Enforce this document — each member owns their K8s manifests	All
Late integration causes last-day failures	High	High	Docker Compose integration starts Day 5, K8s Day 7	Member 3
Hardcoded localhost breaks containers	Medium	High	Code review checklist + CI environment tests	All
API contract mismatches between services	Medium	High	Freeze contracts by Day 3, integration tests validate	All
Kafka consumer group conflicts	Medium	Medium	Enforce consumer group naming convention	Member 3
PostgreSQL schema conflicts	Low	High	Each service owns its own database schema	Member 3
CI pipeline fails on delivery day	Medium	High	Run full pipeline on every PR from Day 1	All
Secrets committed to Git	Low	Critical	<code>.env</code> in <code>.gitignore</code> · PR review checklist	Member 1
Docker image too large (slow CI)	Medium	Low	Multi-stage builds · Alpine base images	All
Missing health probes block K8s	Medium	Medium	DoD checklist enforced in code review	Member 3

13.2 Mitigation Actions Timeline

Action	Target Date	Owner
API contracts reviewed and frozen	Day 3	All
Docker Compose infrastructure running	Day 1	Member 3
All services have a working Dockerfile	Day 4	All
First service deployed to Kubernetes	Day 6	Member 1

Action	Target Date	Owner
All services deployed to Kubernetes	Day 10	All
Integration tests pass end-to-end	Day 12	All
Production readiness checklist signed off	Day 13	Member 3

14. Production Readiness Checklist

14.1 Service-Level Checklist (Each Member completes for their services)

Code

- ☐ All features implemented
- ☐ No `TODO` / `FIXME` in production code
- ☐ No hardcoded localhost, IPs, or credentials
- ☐ All config externalized

Testing

- ☐ Unit tests $\geq$ 70% line coverage
- ☐ Integration tests pass with Testcontainers
- ☐ All CI pipeline stages green

Docker

- ☐ Multi-stage Dockerfile
- ☐ `.dockerignore` present
- ☐ Image builds without errors
- ☐ Container starts and passes health check
- ☐ Runs as non-root user

Docker Compose

- ☐ Service block in `docker-compose.yml`
- ☐ Service starts with `docker compose up`
- ☐ Health check passes

Kubernetes

- ☐ `deployment.yaml` with resource limits and probes
- ☐ `service.yaml` correct
- ☐ `configmap.yaml` complete
- ☐ Pod reaches `Running/1/1`
- ☐ Readiness and liveness probes pass

- ☐ `/actuator/health` returns `UP`
- ☐ `/actuator/prometheus` returns metrics
- ☐ Logs clean (no errors on startup)

Security

- ☐ No secrets in code or config files
- ☐ Runs as non-root container user

Documentation

- ☐ `README.md` in service folder
- ☐ Swagger accessible
- ☐ API contract in `docs/contracts/`

14.2 Platform-Level Checklist (Member 3)

- ☐ All shared infrastructure running in Kubernetes
- ☐ All Kafka topics created and verified
- ☐ PostgreSQL schemas initialized for all services
- ☐ Ingress routing all services correctly
- ☐ Prometheus scraping all service `/actuator/prometheus` endpoints
- ☐ Grafana dashboards displaying live data
- ☐ All alerting rules configured
- ☐ Namespace and RBAC configured

15. Final Delivery Checklist

15.1 Pre-Delivery Validation

Category	Item	Owner	Done
Services	All 7 services deployed and <code>Running/1/1</code>	All	☐
Services	All health checks return <code>UP</code>	All	☐
Services	Full API flow verified through Gateway	Member 1	☐
Frontend	Next.js deployed and connected to Gateway	Member 4	☐
Infrastructure	PostgreSQL, Redis, Kafka, Zookeeper healthy	Member 3	☐
Kafka	All event flows verified end-to-end	Member 3	☐
Testing	All unit and integration tests pass	All	☐

Category	Item	Owner	Done
Testing	Full end-to-end user flow tested manually	Member 4	☐
CI/CD	All GitHub Actions pipelines green on <code>main</code>	All	☐
Monitoring	Prometheus collecting metrics from all services	Member 3	☐
Monitoring	Grafana dashboards live and displaying data	Member 4	☐
Security	No secrets present in Git history	All	☐
Docs	All API contracts documented in <code>docs/contracts/</code>	All	☐
Docs	Architecture overview complete	Member 4	☐
Docs	Deployment guide written	Member 3	☐
Docs	Local setup guide written	Member 4	☐
Demo	Demo script and flow prepared	All	☐
Demo	All members can explain their own services	All	☐

15.2 Pre-Demo Validation Script

Run this in order before the demonstration:

```
bash
```

```
#!/bin/bash
```

```
echo "=== TechHub Pre-Demo Validation ==="
```

```
# 1. All pods running
```

```
echo -e "\n[1] Pod Status:"
```

```
kubectl get pods -n techhub
```

```
# 2. All services exposed
```

```
echo -e "\n[2] Services:"
```

```
kubectl get services -n techhub
```

```
# 3. Ingress routes
```

```
echo -e "\n[3] Ingress:"
```

```
kubectl get ingress -n techhub
```

```
# 4. Health check each service
```

```
echo -e "\n[4] Health Checks:"
```

```
for deployment in api-gateway user-service event-service project-service \
    team-service notification-service community-service; do
    status=$(kubectl exec -n techhub deploy/$deployment -- \
        wget -qO- http://localhost/actuator/health 2>/dev/null | \
        python3 -c "import sys,json; print(json.load(sys.stdin)['status'])" 2>/dev/null)
    echo "    $deployment: ${status:-UNKNOWN}"
done
```

```
# 5. Kafka topics
```

```
echo -e "\n[5] Kafka Topics:"
```

```
kubectl exec -n techhub deploy/kafka -- \
    kafka-topics.sh --list --bootstrap-server localhost:9092
```

```
# 6. Prometheus targets
```

```
echo -e "\n[6] Prometheus: Check http://localhost:9090/targets"
```

```
echo -e "\n=== Validation Complete ==="
```

15.3 Demo User Flow (Recommended Sequence)

1. Register a new user (User Service via Gateway)
2. Login and receive JWT token (User Service via Gateway)
3. Create a new Event (Event Service via Gateway)
 - Verify notification event triggered via Kafka
4. Create a Project (Project Service)
5. Create a Team and add members (Team Service)
 - Verify member-joined notification
6. Publish a Community post (Community Service)

7. View notification history (Notification Service)

8. Show Grafana dashboard – metrics visible during the above flow

9. Show Prometheus – scraping all services

10. Show GitHub Actions – all pipelines green

Document 03 of 03 — TechHub SRE Governance Suite Owner: Member 3 (SRE / Platform Lead) · Member 4 (QA & Documentation Lead)

Document Suite Index

Document	File	Focus
Doc 01	01_GIT_AND_GITHUB_GOVERNANCE.md	Repository structure · Branching · Commits · PRs · GitHub Actions · Tracking
Doc 02	02_DEVOPS_K8S_DOCKER_GOVERNANCE.md	Docker · Kubernetes · Compose · Deployment · Redis · Health Checks
Doc 03	03_SRE_STANDARDS_OPERATIONS.md	Kafka · API Contracts · Monitoring · Logging · Security · Testing · Delivery

TechHub Platform Engineering — Internal SRE Governance Suite